QVT-based SysML v2 Model Version Difference Transformation

line 1: 1st Chen Zhou
line 2: *Institute of Computer Application*
line 3: *China Academy of Engineering Physics*
line 4: Mianyang, China
line 5: zhou_chen99@163.com

line 1: 2nd Ling Li
line 2: *Institute of Computer Application*
line 3: *China Academy of Engineering Physics*
line 4: Mianyang, China
line 5: 1139608130@qq.com

line 1: 3rd Qiang Ren
line 2: *Institute of Computer Application*
line 3: *China Academy of Engineering Physics*
line 4: Mianyang, China
line 5: renqiangnwpu@163.com

line 1: 4th Zixin Mu*
line 2: *Institute of Computer Application*
line 3: *China Academy of Engineering Physics*
line 4: Mianyang, China
line 5: 965251198@qq.com

line 1: 5th Bing Yu
line 2: *Institute of Computer Application*
line 3: *China Academy of Engineering Physics*
line 4: Mianyang, China
line 5: hnwhyb@163.com

line 1: 6nd Baoran An
line 2: *Institute of Computer Application*
line 3: *China Academy of Engineering Physics*
line 4: Mianyang, China
line 5: anbaoran@qq.com

***Abstract*—In order to address the issue of model version incompatibility in supporting the implementation of Model-Based Systems Engineering (MBSE) with SysML v2, this paper first elucidates a feasible engineering modelling method based on the SysML v2 specification. Subsequently, leveraging model transformation concepts, a v2 model version conversion method based on the QVT Relations Language is proposed. Various potential compatibility scenarios and implementation paths are meticulously analysed. On this foundation, a design scheme for program modules ensuring version full transitivity is presented. Finally, the proposed methods and programs are validated through case studies involving version conversion under two different scenarios. The results indicate that the proposed approach effectively resolves SysML v2 model version compatibility issues arising from changes in syntax or semantic patterns.**

Keywords—QVT, SysML v2, MBSE, Model Transformation

I. INTRODUCTION

Model-Based Systems Engineering (MBSE) is currently the core methodology in the industry for achieving top-down design and full lifecycle digitalization of complex products [1]. Its essence lies in using models instead of documents to represent various aspects of the system, ensuring the consistency of information transfer across different stages, platforms, and personnel, thereby enhancing the efficiency and quality of system engineering implementation and supporting the full life cycle management of the system. Consequently, a standardized and robust modeling language is crucial for advancing MBSE, and the *System Modeling Language (SysML)*, which supports the specification, design, analysis, and verification of systems, as well as the modeling of hardware, equipment, software, data, personnel, procedures, and facilities, is currently the most commonly used modeling language in MBSE practice [2].

SysML v1 is a graphical modeling language developed by the *International Council on Systems Engineering (INCOSE)* in collaboration with the *Object Management Group (OMG)* based on the *Unified Modeling Language (UML)* and *Meta-Object Facility (MOF)* standards, boasts extensive modeling capabilities and intuitive model representation. It is the mainstream version used in the industry today. However, SysML v1 has certain limitations in complex system development, including imprecise semantics, lack of formal syntax, and insufficient capability to express requirements and structure, rendering it less suitable for handling highly complex systems [3].

To address these issues, the OMG began the development of SysML v2 in 2017, aiming to improve SysML v1 in terms of language precision, conceptual consistency, interoperability, usability, and extensibility, as well as providing a migration path for SysML v1 users and implementers, thereby establishing SysML v2 as the next-generation systems modeling language to further enhance its application and effectiveness in MBSE practice [4]. In March 2023, the *SysML v2 Submission Team (SST)* released a preliminary version that has initially demonstrated engineering implementation capabilities. The complete SysML v2 specifications, metamodels, example models, and experimental tools can be accessed from its official GitHub repository.

To explore superior MBSE system modeling solutions, SysML v2 was adopted as the system modeling language in our latest engineering projects. Based on the preliminary release version's related materials, we conducted engineering experiments with SysML v2. As a result, we have developed a relatively complete SysML v2 toolkit that supports textual modeling, graphical modeling, expression execution, diagram generation, serialization, and compatibility with v1 models. Throughout this process, a substantial number of valuable v2 models have been accumulated, and SysML v2 has demonstrated significant advantages in terms of formalization, understandability, and extensibility. However, using an evolving language as the foundation for engineering implementation also presented several challenges. Since SysML v2 is still in its early development stage, its core undergoes continuous monthly updates, which include enhancements to syntactic concepts and bug fixes. Consequently, our tool's language core must continuously keep pace with these updates. Additionally, due to the scarcity of application cases for SysML v2, new or modified engineering modeling requirements often necessitate changes

2024 China Automation Congress (CAC) | 979-8-3503-6860-4/24/$31.00 ©2024 IEEE | DOI: 10.1109/CAC63892.2024.10864958

or additions to semantic patterns, requiring corresponding adjustments to the tool architecture. The frequent changes in SysML syntax, semantics, and tool architecture lead to rapid discrepancies between new and old model data, making it difficult to ensure compatibility of constructed models across different tool versions, which severely affects user trust in the tools and model data. From a software engineering perspective, this issue is a typical data compatibility problem, where software tools should have the ability to be compatible with multiple versions of data. Depending on the capability, compatibility can generally be classified into forward compatibility, backward compatibility, and full compatibility [5]. Each type also needs to consider whether it possesses transitivity, which means compatibility should not be limited to adjacent versions. Solving compatibility issues requires addressing both software interfaces and data layers. For modeling software, consistency of information across different model versions is essential, with differences primarily reflected in the data form. To achieve full-transitive compatibility, one feasible approach is to convert the models from a data perspective to ensure they are compatible with the software architecture version. The core of this method is model transformation. In systems engineering, model transformation refers to the process of transferring information between different models, where the source and target models' languages differ in terms of type, abstraction level, and formalization degree. The key is language transformation [6]. Here, although the languages used by different version models are all SysML v2, they differ in syntax and semantic patterns, making this a typical model transformation problem.

As previously mentioned, SysML v2 is implemented based on the MOF specification. For models compliant with the MOF standard, the OMG has defined a set of standards dedicated to transformations, queries, and mappings between models conforming to the MOF standard, known as *Query/View/Transformation (QVT)*. In addition, *Atlas Transformation Language (ATL)*, which is highly integrated with *Eclipse Modeling Framework (EMF)*, is also used to support the transformation of MOF system models. Both QVT and ATL define a set of formalized syntaxes for transformation rules to describe these rules at the metamodel level. They also employ *Object Constraint Language (OCL)* to aid in defining transformation conditions and constraints. The primary difference between the two lies in their support for transformation rule definitions: QVT supports bidirectional transformation rules, whereas ATL only supports unidirectional transformation. Therefore, from the perspective of achieving full compatibility, this paper will explore the use of QVT-based methods to realize the transformation between different versions of SysML v2 models.

This paper is organized as follows: In Section 2, the establishment of SysML v2 models and the approach to v1-like modeling methods are introduced. Section 3 presents the QVT-based model version conversion method and program framework. In Section 4, two case studies are used to experimentally analyze the proposed methods, and Section 5 offers conclusions.

II. System Modelling Method Based on SysML v2

In MBSE practice, the establishment of system models relies on the support of modeling languages, methods, and tools [2]. Modeling methods define the purpose, scope, and process sequence of system construction and are tailored to specific modeling languages or tools. For SysML v1, common modeling methods include MagicGrid, Harmony-SE, and Arcadia, etc. These methods rely on SysML v1's graphical modeling syntax to construct the required system model through a specific sequence of views and related elements.

For SysML v2, its precise syntax and formal expression capabilities allow for both textual and graphical modeling. The latest SysML v2 specifications provide detailed textual and graphical syntax, with the textual syntax being largely complete while the graphical syntax is still maturing.

Using the textual syntax, a syntactically correct SysML v2 textual model, as shown in Fig. 1, can be constructed. This model can be further converted into memory instances using the parsing programs provided by the official sources.

Part Definition Example.sysml

```
package 'Part Definition Example' {
        import ScalarValues::*;

        part def Vehicle {
                attribute mass : Real;
                attribute status : VehicleStatus;

                part eng : Engine;

                ref part driver : Person;
        }

        attribute def VehicleStatus {
                gearSetting : Integer;
                acceleratorPosition : Real;
        }

        part def Engine;
        part def Person;
}
```

Fig. 1. SysML v2 example model

On the other hand, since SysML v2 has officially provided an implementation based on XText and the *Eclipse Modeling Framework (EMF)*, which includes XText-formatted v2 syntax definitions and Ecore metamodel files and Java artifacts generated based on them and EMF, these Java artifacts contain factory classes for creating instances of SysML v2 elements and various auto-generated utility classes. Thus, by leveraging these generated Java artifacts, v2 models can be conveniently created directly in memory via programming [7].

Both aforementioned methods can construct syntactically correct SysML v2 models. However, there is no publicly available research on how to appropriately and systematically use v2 syntax to describe arbitrary systems, i.e., modeling methods oriented towards SysML v2.

Given SysML v2's design focus on backward compatibility with v1, an engineeringly feasible approach is to use v2 to express v1 syntax. This allows for the construction of models using a v1-like modeling method while retaining the formalization and other capabilities of v2. The key to this approach is to clearly define the mapping rules between SysML v1 and v2 syntax.

For the transformation between SysML v1 and v2, the SST has provided an official solution in its latest release, offering a v1-to-v2 transformation rules document suitable for the EMF model transformation module [8]. This document describes the mapping from stereotypes and the UML types used in SysML v1.7 to the SysML v2 metamodel, precisely defining the calculation rules for type parameters

using OCL. Additionally, it ensures the completeness and accuracy of the transformation through some additional constraint expressions and specific mapping data structures. The core idea of the specification is to utilize the overall semantic mapping nature of the metamodel structures between the two versions, as illustrated in Fig. 2.

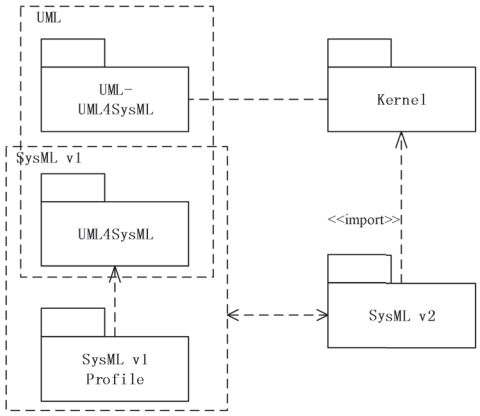

Fig. 2. SysML v1 and v2 kernel structure overview and mapping

As shown in Fig. 2, SysML v1 essentially serves as a standard extension library of UML, forming a metamodel collection in conjunction with the UML subset for systems engineering (UML4SysML) and SysML extension library elements. In contrast, SysML v2 completely excludes any association with the UML language, utilizing a new *Kernel Modeling Language (KerML)* as its metamodel foundation. Compared to UML, KerML is a platform-independent, more lightweight modeling language with more precise syntax and semantics, completely independent of the systems engineering domain. The SysML v2 language is specialized based on KerML. Therefore, considering the overall mapping perspective, v1, based on UML, can be compared to v2, based on KerML, where the union of UML4SysML and SysML extension library elements has corresponding relationships in semantics with most elements in the v2 element set. Table 1 shows some of the type mappings provided by the SST in their document.

TABLE I. TABLE OF PARTIAL ELEMENT TYPE MAPPING IN THE SST TRANSFORMATION DOCUMENT

SysML v1	SysML v2
part property / block	part / part def
value property / value type	attribute / attribute def
proxy port / interface block	port / port def
action / activity	action / action def
state / state machine	state / state def
requirement	requirement def
connector / association block	connection / connection def

As shown in the table, the similarity between elements of the two versions is also reflected in the comparison of type names at both ends of the mapping, such as the v1 stereotype *Requirement* being mapped to *RequirementDefinition*, and *ConstraintBlock* being mapped to *ConstraintDefinition*.

However, it should be noted that although the SysML v2 metamodel inherits some of the related semantics from v1, its more compact language structure and more general metamodel system introduce significant language differences. The numerous combinations of metamodel types and attributes across the two versions add substantial complexity to precise mapping. Consequently, the current official document only provides mapping rules for some v1 types to v2, and does not yet fully cover the basic feature set elements of SysML v1. For these elements, the v2 syntax offers two enforced mapping methods:

a)Extending v2 usable element types through specialization

v2 builds domain libraries through *Specialization* relationships. For v1 types lacking mappings in the transformation specification, corresponding extended types can be constructed for enforced mapping through semantic analysis and *Specialization* relationships. For instance, various specialized stereotype mappings of *Requirement* can be used to construct various extended requirement element types in v2 to support mapping, as illustrated in Fig. 3.

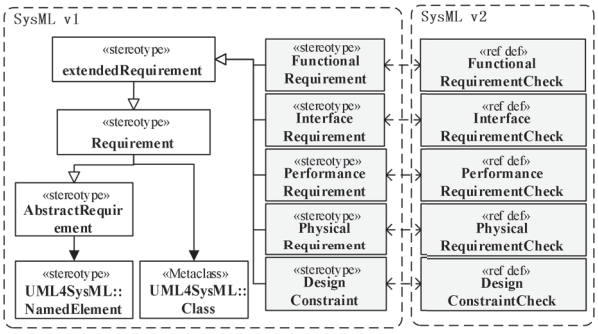

Fig. 3. Extension and mapping of SysML v2 element types based on specialisation mechanism

b) Extending v2 types through metadata

In addition to Specialization relationships, SysML v2 also offers a language extension approach based on metadata types (MetaClass), primarily supported by the *MetadataDefinition* and *MetadataUsage* metatypes in v2.

Using the above methods can fully support the mapping and conversion between SysML v1 and v2, thereby enabling the use of v1-like modeling methods to construct v2 system models in MBSE practice. As shown in Fig. 4, the left side of the figure displays the v2 textual representation tree of the model, while the right side shows the v1-like view.

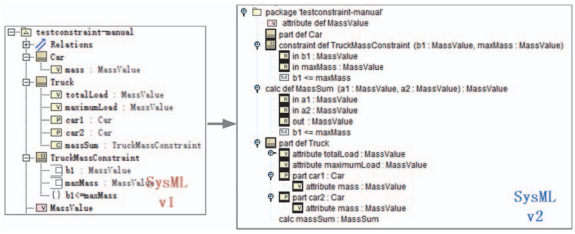

Fig. 4. Example of v1-like modelling

III. QVT-BASED MODEL VERSION SWITCHING

A. QVT Specification

QVT, as a transformation description language specifically for MOF models, is itself defined as a model with metamodels conforming to the MOF standard. Along with these metamodels, the concrete syntax of QVT is also defined, supporting the description of transformation rules for source and target models at the metamodel level and executing transformations at the instance level. The overall approach is illustrated in Fig. 5.

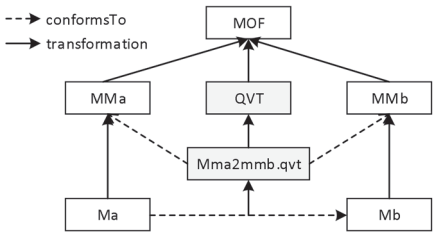

Fig. 5. Overview of the QVT Transformation

The QVT specification supports both declarative and imperative rule description methods, providing three descriptive languages: *QVT-Relations (QVTr)*, *QVT-Core (QVTc)*, and *QVT-Operational (QVTo)*. The first two belong to declarative languages, while QVTo is an imperative language. Any of these languages can be used to complete the transformation description. Relatively speaking, QVTr is more user-friendly and superior in expressiveness compared to QVTc, and it can nest QVTo, making it the most commonly used.

In QVTr, transformations between models are represented through a series of relations, with each relation representing a binary transformation rule. Relations are divided into top-level and non-top-level relations, with top-level relations serving as the entry point for transformations and being executed by default, while non-top-level relations are executed when invoked.

A relation consists of several variable definitions, domain definitions, and a pair of when and where predicate statements. The domain is associated with a model involved in the transformation, searching for elements in the model that meet the conditions. The when clause specifies the necessary conditions for the relation to hold, while the where clause specifies additional constraints that the associated elements need to meet [9]. A typical QVTr transformation code is shown in Fig. 6.

QVTr Example.qvtr

```
import SimpleUML './UML.xmi';
import SimpleRDBMS './RDBMS.xmi';
transformation umlRdbms (uml : SimpleUML, rdbms : SimpleRDBMS) {
    top relation PackageToSchema {
        checkonly domain uml p:Package {name=pn};
        enforce domain rdbms s:Schema {name=pn};
    }

    top relation ClassToTable {
        checkonly domain uml c:Class {
            namespace=p:Package{},
            kind='Persistent',
            name=cn
        };
        enforce domain rdbms t:Table {…};
        when {
            PackageToSchema(p, s);
        }
        where {
            AttributeToColumn(c, t);
        }
    }

    relation AttributeToColumn {…}
}
```

Fig. 6. QVT Relations example

In the execution semantics of QVTr, when the target model is specified, a relation holds if, for each source model element that matches the source domain pattern and satisfies the when clause after binding its information to the relation variables, there exists a target model element that matches the target domain pattern and satisfies the where clause after binding its information to the remaining variables. Conversely, if the relation does not hold, operations are executed based on the overall execution mode of the transformation and the modifier of the target domain, as shown in Table 2 [10].

TABLE II. EXECUTION SEMANTICS OF DIFFERENT TRANSFORMATION EXECUTION MODES AND DOMAIN MODIFIERS

Trans Exec Mode	Source Domain Modifier	Target Domain Modifier	Exec Results
check only	checkonly /enforce/ None	checkonly /enforce/ None	Checks whether the relations hold in all directions and returns the results of the check without modifying the model
enforce	checkonly /enforce	enforce	For each matching source domain element, if there is no matching target domain element, the target element is created or modified. For each matching target domain element, if there is no matching domain element, delete the target element
	None	enforce	For each matching source domain element, if there is no matching target domain element, the target element is created or modified.
	checkonly /enforce/ None	checkonly	Checks only the target elements without modifying the target model

For a complete QVTr transformation as illustrated in Fig. 6, the execution process begins with the top-level relation. It performs a consistency check between the source and target model information and completes the addition, deletion, or update of target model elements, thereby achieving the transformation from the source to the target model.

B. SysML v2 Model Version Transformation

As previously mentioned, SysML v2 models undergo version changes in two scenarios: updates to the SysML v2 specification and semantic pattern changes during engineering use. The QVTr-based model transformation approach for these scenarios will be discussed separately.

For the first scenario, since the release of the preliminary version, the SST has submitted several updates to the v2 syntax specification. These updates are mainly focused on the abstract syntax and graphical concrete syntax, with the textual concrete syntax remaining relatively stable. This results in v2 textual models describing the same information and structure having different *Abstract Syntax Trees (AST)* after parsing. When saving the model from an in-memory model to XMI or other formats, significant differences arise.

As this scenario involves changes to the v2 metamodel, models from different versions are essentially constructed using two similar modeling languages. When using QVTr to describe the transformation between these versions, both versions of the SysML v2 metamodel must be introduced for heterogeneous model transformation, as illustrated in the transformation definition in Fig. 7.

Differ-place SysML v2 Model Trans.qvtr

```
import SysMLv2-X './SysML-v2(Release X)';
import SysMLv2-Y './SysML-v2(Release Y)';
transformation Sysmlv2MappingVerNVerM
(sysmlOld : SysMLv2-X, sysmlNew : SysMLv2-Y)
{…}
```

Fig. 7. Differ-place transformation definition

Next, relations are used to define the rules involved in the aforementioned transformation. Since the source and target models are similar, both copy transformation and differential transformation need to be considered. Copy transformation is achieved by invoking the built-in operations *DoElementCopy* and *DoElementPropertyCopy* through the where clause, while differential transformation is implemented using domain pattern mapping based on specific conditions. The version differences caused by syntax changes mainly include element type transformations and attribute changes. Examples of type changes include *Import* to *NamespaceImport*, *SourceEnd*/*TargetEnd* to *Feature*, and *ItemFlowFeature* to *ReferenceUsage*, while attribute changes include *humanId* to *shortName* and *declaredName* to *name*. Fig. 8 shows the relation definitions for these two types of differences.

Type Change Relations

```
relation MappingType {
    varId: String;
    enforce domain sysmlOld
s:OldType {
        id=varId
    };
    enforce domain sysmlNew
t:NewType {
        id=varId
    };
    where {

    DoElementPropertyCopy(s,
t);
    }
}
```

Attr Change Relations

```
relation MappingAttr {
    varId: String;
    varAttrValue: OclAny;
    enforce domain sysmlOld s:Type1 {
        id=varId,
        <AttrName>=varAttrValue
    };
    enforce domain sysmlNew t:Type1 {
        id=varId,

    <AttrName>=OCLExpression(varAttrValue)
    };
    where {
        DoElementPropertyCopy(s, t);
    }
}
```

Fig. 8. Type change and attribute change relations

For the second scenario, due to the lack of SysML v2 modeling methods and the evolution of engineering modeling standards, the patterns for the same information will continuously evolve to align with the latest standards, resulting in model version differences. This process does not involve changes to the SysML v2 syntax, and both the old and new models are based on the same metamodel version. Accordingly, in the QVTr transformation definition, only one type of model needs to be introduced. Furthermore, the in-place transformation mechanism of QVTr can be utilized, where both the source and target models are bound to the same in-memory model instance, allowing for in-place operations. Fig. 9 shows the transformation definition for this scenario.

In-place SysML v2 Model Trans.qvtr

```
import SysMLv2-X './SysML-v2(Release X)';
transformation Sysmlv2MappingVerNVerM
(sysmlOld : SysMLv2-X, sysmlNew : SysMLv2-X)
{…}
/* sysmlold and sysmlNew are bound to the
same model instance */
```

Fig. 9. In-place transformations definition

Similarly, relations are used to define the transformation rules for this scenario. Since the source and target models are the same, only the rules addressing the differences need to be considered. The differences due to changes in the semantic patterns mainly involve modifications to element attributes and nested structures, such as changes in element extension patterns. The relevant relation definitions are shown in Fig. 10.

Nested structure change relations

```
relation MappingNestedStructExp {
    varId: String;
    enforce domain sysmlOld s:Type1 {
        id=varId
    };
    enforce domain sysmlNew t:Type1 {
        id=varId,
        <nestedAttrName>= n1:TypeN{...}
    };
}
```

Fig. 10. Nested structure change relations

C. SysML v2 full transitive versioning module design

Based on the aforementioned methods, the version management module for SysML v2 models can be designed as shown in Fig. 11, thereby supporting full-transitive compatibility of v2 model versions.

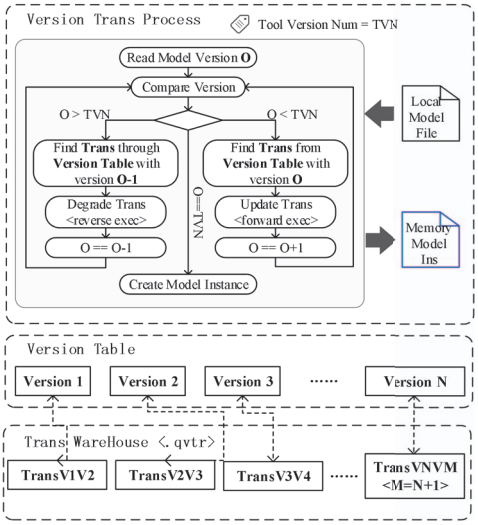

Fig. 11. Full transitive versioning module overview

The main approach of this module is to add version serial number information in the model metadata, establish a QVTr version conversion library, and create a one-to-one mapping relationship between version numbers and conversion files. By leveraging QVTr's support for bidirectional conversion, model version upgrades and downgrades are supported. Additionally, through continuous version numbers and a comprehensive rule library, the transitivity of version conversions is ensured, achieving a full-transitive compatible v2 model version system.

IV. Case Study

The following will present two case studies to validate and analyze the proposed methods.

a) SysML v2 Core Upgrade Case

Taking the change in the *Import* element type following the specification update as an example, in the SST releases from 2022-10 onwards, the *Import* element type has been subdivided into *NamespaceImport* and *MembershipImport*. The *Import* element has become an abstract type, and in older versions, the *Import* semantics pertain to namespace importation, thus necessitating its replacement with the *NamespaceImport* type. The QVTr code for this difference transformation is shown in Fig. 12.

Differ-place SysML v2 Model Trans.qvtr

```
import SysMLv2-202210 './SysML-v2(Release 202210)';
import SysMLv2-202211 './SysML-v2(Release 202211)';
transformation Sysmlv2MappingVerNVerM (sysmlOld : SysMLv2-202210, sysmlNew : SysMLv2-
202211) {
    key Element{id};
    top relation ElementMapping{
        enforce domain sysmlOld s:Element{};
        enforce domain sysmlNew t:Element{};
        where {
            MappingImport(s, t) or DoElementCopy(s, t);
        }
    }
    relation MappingImport{
        varId: String;
        enforce domain sysmlOld sImport:Import {
            id=varId
        };
        enforce domain sysmlNew tNamespaceImport:SysMLv2-202211::NamespaceImport {
            id=varId
        };
        where {
            DoElementPropertyCopy(sImport, tNamespaceImport);
        }
    }
}
```

Fig. 12. Differ-place transformation example

The upgrade results are shown in Fig. 13.

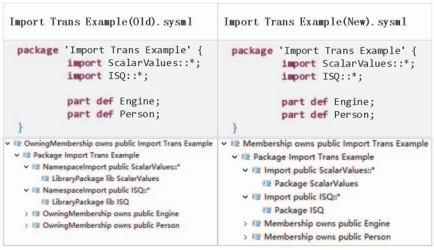

Fig. 13. Differ-place transformation result

As shown in the figure, the text models of the two versions remain the same, but the parsed model tree structure has changed.

b) Case of Modeling Semantic Pattern Change

Taking the change in element extension patterns as an example, it was mentioned that SysML v2 can extend element types through *Specialization* relationships and metadata. This requires switching based on project needs. In the scenario of using v2 to express v1, the extension of *Requirement*-related elements can be changed from the specialization mechanism to the metadata mechanism. The relevant QVTr code is shown in Fig. 14.

In-place SysML v2 Model Trans.qvtr

```
import SysMLv2-202405 './SysML-v2(Release 202405)';
transformation Sysmlv2MappingVerNVerM (sysmlOld : SysMLv2-202405, sysmlNew : SysMLv2-
202405) {
    key Element{id};
    top relation ElementMapping{
        enforce domain sysmlOld s:Element{};
        enforce domain sysmlNew t:Element{};
        where {
            MappingFunctionalRequirementCheck(s, t) or ... or
MappingRequirementCheck(s, t);
        }
    }
    relation MappingFunctionalRequirementCheck{
        varId: String;
        enforce domain sysmlOld sReq:RequirementDefinition {
            id=varId,
            ownedSubclassifier=subclassifier:Subclassification {
                superclassifier=super:Classifier{
                    name='FunctionalRequirementCheck'
                }
            }
        };
        enforce domain sysmlNew tReq:RequirementDefinition {
            id=varId,
            ownedMetadata=metadata:MetadataUsage{
                definition='sysml:functionalRequirement'
            }
        };
    }
    ......
    relation MappingRequirementCheck{
        varId: String;
        enforce domain sysmlOld sReq:RequirementDefinition {
            id=varId
        };
        enforce domain sysmlNew tReq:RequirementDefinition {
            id=varId,
            ownedMetadata=metadata:MetadataUsage{
                definition='sysml::Requirement'
            }
        };
    }
}
```

Fig. 14. In-place transformation result

The upgrade results are shown in Fig. 15.

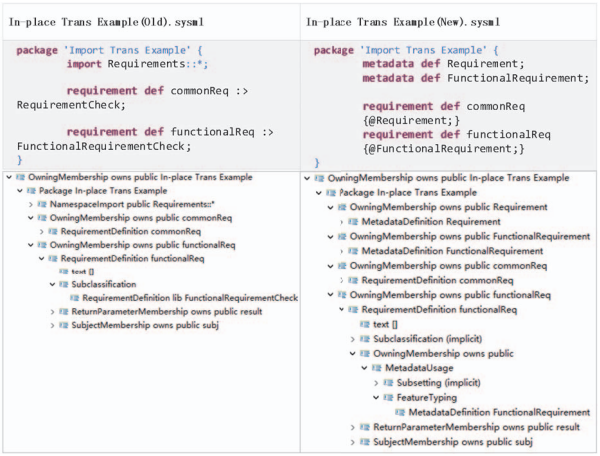

Fig. 15. In-place transformation result

From the figure, it can be observed that both the v2 text models and the parsed model tree structures have undergone changes before and after the transformation.

V. Conclusion

In this paper, the features and engineering modelling methods of SysML v2 are presented, and the concept of model transformation is applied to solve the version compatibility problem of v2 models, giving the method of v2 model version conversion based on QVT Relations, and various potential compatibility scenarios and implementation paths are meticulously analysed. Based on this, a design scheme for program modules ensuring version full transitivity is presented. Finally, the proposed methods and program are analyzed and validated through version conversion cases under two different scenarios, demonstrating that the methods can effectively resolve version compatibility issues in v2 models caused by changes in syntax or semantic patterns.

References

[1] L. Z., W. G., L. J., G. B. D., K. D., and Y. Y., "Bibliometric Analysis of Model-Based Systems Engineering: Past, Current, and Future," IEEE Transactions on Engineering Management, vol. 71, pp. 2475-2492, 2024/1/1 2024. 10.1109/TEM.2022.3186637

[2] S. Friedenthal, A. Moore and R. Steiner, "Chapter 2 - Model-Based Systems Engineering," in A Practical Guide to SysML (Third Edition), S. Friedenthal, A. Moore and R. Steiner, Eds. Boston: Morgan Kaufmann, 2015, pp. 15-29. https://doi.org/10.1016/B978-0-12-800202-5.00002-3

[3] M. Schacher, "The Limits of SysML v1 and how SysML v2 addresses them (1),", 2023.

[4] M. Schacher, "The Limits of SysML v1 and how SysML v2 addresses them (2),", 2023.

[5] CONFLUENT, "Schema Evolution and Compatibility for Schema Registry on Confluent Platform,", 2024.

[6] Y. Guo, J. Wei, G. Chen, and S. She, "A Unified Model-Based Systems Engineering Framework Supporting System Design Platform Based on Data Exchange Mechanisms,", Virtual, Online, 2022, pp. 99-112. 10.1007/978-981-19-3610-4_7

[7] L. Bettini, "Implementing Domain-Specific Languages with Xtext and Xtend,", 2013.

[8] OMG, "OMG Systems Modeling Language (SysML) Part 2: SysML v1 to SysML v2

[9] Transformation,", Version 2.0 Beta 2 ed, 2024.

[10] OMG, "Meta Object Facility (MOF) 2.0 Query/View/Transformation Specification,", 2016.